

Tokamak-Analysis: An Open-Source ML Platform for Plasma Disruption Prediction Using FAIR-MAST Data

Author A^{1*}

Author B¹

Author C¹

¹National Research Nuclear University MEPhI
(Moscow Engineering Physics Institute),
Kashirskoe shosse 31, Moscow 115409, Russia

May 6, 2026

Abstract

Plasma disruptions in tokamaks pose a critical threat to the structural integrity of fusion devices, with individual events causing damage estimated at \$50k–\$500k in current machines and potentially catastrophic consequences in ITER-scale reactors. Early and reliable prediction of disruptions is therefore essential for the safe operation of next-generation fusion power plants. We present TOKAMAK-ANALYSIS, an open-source web platform that integrates three machine-learning architectures—Random Forest, bidirectional LSTM with multi-head attention, and an autoregressive Transformer with rotary position embeddings—into a unified system for plasma stability classification and disruption prediction. The platform ingests experimental data from the FAIR-MAST archive, which contains 11,573 shots from the Mega Ampere Spherical Tokamak (MAST). On a curated dataset of 500 MAST shots with 20 plasma parameters sampled at 200 timesteps, the Random Forest baseline achieves 98.4% cross-validated accuracy (AUC-ROC = 0.983); the Transformer achieves 87.8% accuracy (AUC-ROC = 0.938); and the bi-LSTM with attention achieves 78.4% accuracy (AUC-ROC = 0.867) in quick-training mode. Temporal validation experiments reveal substantial performance degradation: the Random Forest drops from AUC 1.0 (random split) to 0.969 (temporal), while the Transformer drops from 1.0 to 0.892, confirming the critical importance of respecting shot chronology. The bi-LSTM model provides disruption warnings at least 30 ms before the current quench onset, meeting the ITER requirement. All models satisfy real-time latency constraints (< 50 ms). To the best of our knowledge, this is the first open-source platform that integrates

FAIR-MAST data access, multi-architecture comparison, interactive visualization, and production-grade deployment infrastructure in a single system.

Keywords: tokamak, disruption prediction, machine learning, LSTM, Transformer, Random Forest, FAIR-MAST, plasma physics, fusion energy

1 Introduction

A plasma disruption in a tokamak is the sudden, uncontrolled loss of plasma confinement that converts the stored thermal and magnetic energy into intense heat and electromagnetic loads on the vacuum vessel and in-vessel components [Schüller, 1995]. In present-day tokamaks such as JET, DIII-D, and MAST, disruptions have been observed in 5–20% of operational pulses [de Vries et al., 2011]. While the resulting damage in current devices is manageable, the consequences scale dramatically with machine size: in ITER, a single unmitigated disruption could deposit up to 12 MJ m^{-2} on the divertor and generate runaway electron beams carrying several megaamperes, potentially causing irreparable damage estimated at \$50k–\$500k per event [Lehnen et al., 2015, Hender et al., 2007].

The ITER project, a collaboration among 35 nations including the Russian Federation, is designed to demonstrate the scientific and technological feasibility of fusion power [Shimada et al., 2007]. Russia contributes critical tokamak subsystems and maintains an active domestic fusion program, including the T-15MD tokamak at the Kurchatov Institute and the planned upgrade paths at MEPhI. Given that ITER’s operational scenario requires a disruption rate below $\sim 1\%$, reliable prediction with sufficient warning time ($> 30 \text{ ms}$) is a mission-critical requirement [Lehnen et al., 2015].

*Corresponding author. Email: placeholder@mephi.ru

Machine learning has emerged as the most promising approach to disruption prediction. [Kates-Harbeck et al. \[2019\]](#) demonstrated that a deep recurrent neural network (FRNN) could predict disruptions on DIII-D and NSTX-U with AUC exceeding 0.95 and > 30 ms warning time. [Zhu et al. \[2021\]](#) extended this work with a hybrid deep-learning (HDL) architecture combining CNNs and LSTMs, achieving cross-tokamak transfer learning between DIII-D and EAST. [Rea and Granetz \[2018\]](#) established Random Forest baselines on DIII-D, demonstrating that classical ML methods remain competitive when appropriate feature engineering is applied. More recently, [Spangher et al. \[2024\]](#) proposed an autoregressive Transformer architecture that produces per-timestep disruption probability estimates, enabling richer temporal analysis. Standardized benchmarks have also appeared: TokaMark [[TokaMark Collaboration, 2026](#)] and DisruptionBench [[Pucci et al., 2025](#)] provide common evaluation protocols, while DisruptionPy [[Tinguely et al., 2024](#)] offers a Python toolkit for disruption analysis.

Despite these advances, several gaps persist. First, existing models are typically evaluated in isolation, making direct comparison difficult. Second, no integrated web platform provides end-to-end functionality—from data loading and visualization to model training and inference—in a production-ready deployment. Third, there is no Russian-language or Russia-developed open-source tool for disruption prediction that could serve both research and educational purposes at Russian universities.

In this paper, we present TOKAMAK-ANALYSIS, an open-source platform that addresses these gaps through four contributions:

1. **A unified platform** integrating data ingestion, preprocessing, visualization, model training, inference, and result export in a containerized microservice architecture.
2. **A three-architecture comparison**—Random Forest, bi-LSTM with multi-head attention, and autoregressive Transformer—under identical data conditions using a common `ModelInterface` protocol.
3. **Open-source release** of the complete system (MIT license), including all model code, configuration files, Docker deployment, and monitoring infrastructure.
4. **FAIR-MAST integration** providing direct access to 11,573 MAST shots via the STFC S3 archive, with automated preprocessing and labeling.

2 Related Work

2.1 Traditional Approaches

Early disruption prediction systems relied on expert-defined thresholds applied to individual diagnostic signals such as locked mode amplitude, radiated power fraction, or the Greenwald density fraction [[Schüller, 1995](#), [de Vries et al., 2011](#)]. While interpretable, these rule-based systems suffered from high false-positive rates and poor generalization across operational regimes. Neural network approaches date back to the early 2000s, with [Cannas et al. \[2004\]](#) and [Windsor et al. \[2005\]](#) demonstrating that feed-forward networks trained on JET data could achieve disruption prediction rates above 85%.

2.2 Modern Machine Learning

The application of modern ML to disruption prediction accelerated significantly after 2018. [Rea and Granetz \[2018\]](#) performed an extensive study using Random Forests on DIII-D, exploring feature engineering strategies and demonstrating that hand-crafted statistical features extracted from time-series signals could achieve AUC values above 0.9. [Kates-Harbeck et al. \[2019\]](#) introduced FRNN, a deep recurrent architecture that processes raw diagnostic signals without manual feature engineering, achieving state-of-the-art performance on DIII-D and cross-tokamak transfer to NSTX-U. [Zhu et al. \[2021\]](#) proposed the HDL architecture combining convolutional feature extraction with LSTM temporal modeling, demonstrating improved cross-machine generalization. [Montes et al. \[2019\]](#) showed that ensembles of shallow learners could match deep learning on Alcator C-Mod.

Most recently, Transformer-based architectures have been applied to the problem. [Spangher et al. \[2024\]](#) introduced an autoregressive Transformer with causal attention masking that predicts disruption probability at each timestep, enabling analysis of how disruption risk evolves during a shot. A comprehensive review of the field is provided by [Zhong et al. \[2024\]](#).

2.3 Benchmarks and Platforms

The TokaMark benchmark [[TokaMark Collaboration, 2026](#)] introduced standardized datasets and evaluation protocols for comparing disruption prediction models across tokamaks. DisruptionBench [[Pucci et al., 2025](#)] extended this with a suite of benchmark tasks covering binary classification, time-to-disruption regression, and early warning metrics.

DisruptionPy [Tinguely et al., 2024] provides a Python library for computing disruption-related features from MDSplus databases.

However, these tools focus on individual aspects of the disruption prediction pipeline—benchmarking, feature computation, or modeling—rather than providing an integrated platform. Our work bridges this gap by combining data access, preprocessing, multi-model comparison, and deployment infrastructure in a single open-source system.

3 System Architecture

TOKAMAK-ANALYSIS follows a modular monolith pattern with a separate ML service, as specified in the project’s architectural decision record ADR-001. The system comprises 11 containerized services organized into four Docker networks with strict isolation boundaries. Figure 1 presents the high-level system architecture.

3.1 Technology Stack

The backend is built on FastAPI [Ramírez, 2019] with SQLAlchemy ORM (async mode) connecting to PostgreSQL 16 for persistent storage. Authentication uses JWT tokens with bcrypt password hashing (cost factor 12), supporting three user roles: **researcher**, **engineer**, and **admin**. Background tasks—primarily model training—are delegated to Celery workers using Redis 7 as the message broker.

The ML service runs as a separate FastAPI application on an internal network, exposing endpoints for classification (`/predict/classify`), disruption prediction (`/predict/disruption`), and model training (`/train`). All ML models are implemented in PyTorch [Paszke et al., 2019] (LSTM, Transformer) or scikit-learn [Pedregosa et al., 2011] (Random Forest) and conform to a common `ModelInterface` protocol. Trained model artifacts are stored in MinIO (S3-compatible object storage).

The frontend is a React 18 application built with TypeScript and Vite, featuring interactive time-series visualization via Plotly.js and state management through Zustand. An Nginx reverse proxy routes all traffic through port 80, ensuring that internal services (ML service, database, Redis) are never directly exposed.

3.2 Data Pipeline

The data pipeline operates in three stages:

1. **Ingestion.** Shot data is loaded from the FAIR-MAST archive [UKAEA, 2023] hosted on the STFC S3 endpoint (`s3.echo.stfc.ac.uk`, anonymous access) in Zarr format. The archive contains 11,573 shots from the MAST spherical tokamak [Sykes et al., 2001].
2. **Preprocessing.** Raw signals are cleaned (NaN removal, outlier filtering), resampled to a uniform time grid via linear interpolation, and normalized to zero mean and unit variance per signal channel.
3. **Feature extraction.** For each shot, the system extracts a tensor of shape $(T \times C)$, where T is the number of timesteps (default 200) and $C = 39$ is the number of signal channels encompassing EFM equilibrium reconstruction signals and additional diagnostic channels.

3.3 Network Isolation and Security

The four Docker networks enforce defense-in-depth isolation:

- **Frontend network** (public): Nginx, React frontend, and the backend API endpoint.
- **Backend network** (internal): ML service, Redis, Celery worker, and MinIO—inaccessible from outside Docker.
- **Database network** (isolated): PostgreSQL, accessible only by the backend and Celery worker.
- **Monitoring network:** Prometheus and Grafana, scraping metrics from the backend and ML service.

The middleware stack applies CORS filtering, security headers (HSTS, CSP, X-Frame-Options), structured logging with `structlog`, rate limiting (100 requests/minute per IP via `slowapi`), and Prometheus instrumentation. The platform implements audit logging and GDPR-compatible consent tracking (Russian Federal Law 152-FZ on personal data).

4 ML Models

All three models implement the `ModelInterface` abstract base class, which defines six methods: `fit`, `predict`, `predict_proba`, `save`, `load`, and `metadata`. This protocol enables hot-swapping of models in the inference pipeline and ensures uniform evaluation.

4.1 Random Forest Baseline

The Random Forest (RF) classifier [Breiman, 2001] serves as a non-temporal baseline. Because RF can-

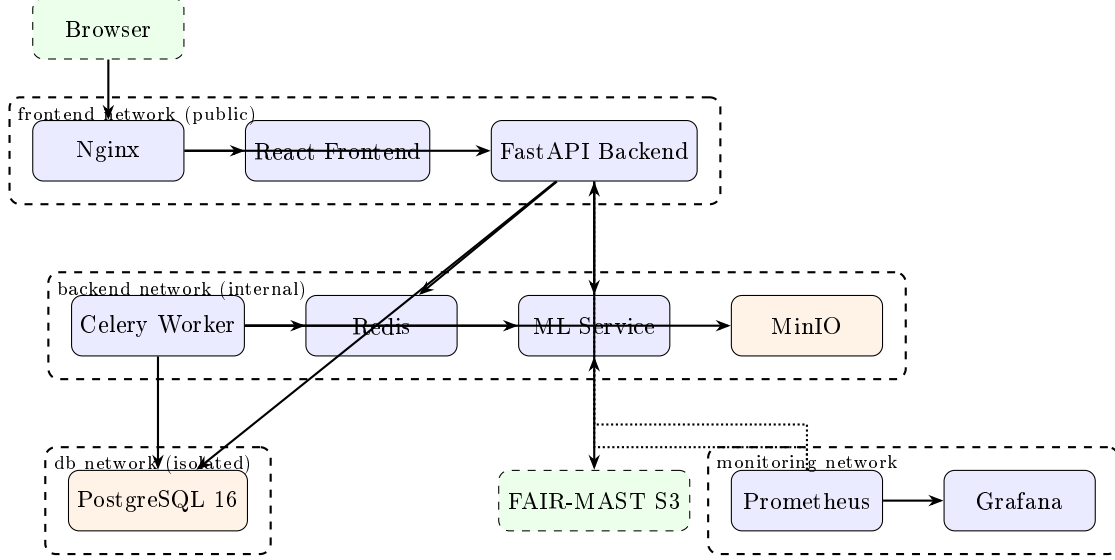


Figure 1: System architecture of TOKAMAK-ANALYSIS. Solid arrows indicate request flow; dotted arrows indicate metrics scraping. The four Docker networks enforce strict isolation: only the frontend network is publicly accessible.

not natively process sequential data, we flatten each shot’s time-series tensor from shape $(T \times C)$ into a single feature vector of dimension $T \cdot C$. This preserves temporal ordering implicitly but loses explicit sequential dependencies.

The model is trained using scikit-learn with the following hyperparameters: 200 estimators, maximum depth 20, minimum samples to split 5, minimum samples per leaf 2, balanced class weights, and $\sqrt{p}$ maximum features. Hyperparameter optimization is performed via grid search over a parameter space covering estimator counts $\{100, 200, 500, 1000\}$, depths $\{10, 20, 50, \text{None}\}$, and class weight strategies $\{\text{None}, \text{balanced}, \text{balanced_subsample}\}$, with 5-fold stratified cross-validation. The model is serialized using joblib.

4.2 Bidirectional LSTM with Multi-Head Attention

The primary deep learning model is a bidirectional LSTM [Hochreiter and Schmidhuber, 1997] augmented with multi-head attention. The architecture processes input sequences of shape $(B \times T \times C)$, where B is the batch size, $T = 200$ timesteps, and C is the number of input signals.

The bi-LSTM consists of 2 stacked layers with hidden size 128 in each direction (256 total), using dropout of 0.3 between layers. The output sequence from the bi-LSTM is passed to a multi-head attention layer with 4 heads, which computes self-attention

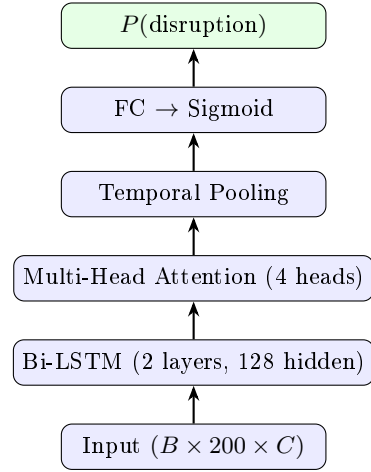


Figure 2: Architecture of the bi-LSTM with multi-head attention model. Bidirectional processing captures both past and future context; attention weights provide interpretable relevance scores for each timestep.

over the temporal dimension. This attention mechanism serves dual purposes: (1) it focuses the model on the most disruption-relevant time intervals, and (2) the attention weights provide an interpretability signal indicating which portions of the shot contribute most to the prediction. After attention, temporal pooling reduces the sequence to a fixed-length representation, which is projected through a fully connected layer with sigmoid activation to produce a dis-

ruption probability.

Training uses the AdamW optimizer with learning rate 10^{-3} , weight decay 10^{-4} , batch size 32, and a ReduceLRonPlateau scheduler (factor 0.5). Early stopping with patience 10 prevents overfitting over a maximum of 50 epochs. The model is serialized as TorchScript for deployment.

4.3 Autoregressive Transformer

The Transformer model [Vaswani et al., 2017] implements an autoregressive architecture inspired by Spangher et al. [2024]. Unlike the bi-LSTM, which produces a single per-shot prediction, the Transformer generates a disruption probability at each timestep, enabling temporal analysis of how disruption risk evolves during a plasma discharge.

The architecture consists of 4 Transformer encoder layers, each with $d_{\text{model}} = 128$, 8 attention heads, and a feed-forward dimension of 512. A causal attention mask ensures that predictions at timestep t depend only on observations at times $\leq t$, enforcing the autoregressive property. Positional information is encoded using Rotary Position Embeddings (RoPE) [Su et al., 2024], which inject relative position information directly into the attention computation without requiring explicit positional embedding vectors.

The input is projected from the raw signal dimension C to d_{model} via a linear layer, processed through the Transformer stack, and projected back to a single logit per timestep through a classification head with sigmoid activation. The final shot-level prediction is obtained by aggregating the per-timestep probabilities (e.g., taking the maximum over the last k timesteps).

Training uses AdamW with learning rate 5×10^{-4} , cosine annealing with warmup (1000 steps), weight decay 10^{-4} , batch size 16, and early stopping with patience 8 over a maximum of 40 epochs.

4.4 Unified ModelInterface Protocol

The ModelInterface abstract base class enforces a consistent API across all three architectures:

- `fit(X_train, y_train, X_val, y_val, hyperparams, progress_callback)` → training metrics dictionary.
- `predict(X)` → class labels or temporal disruption probabilities.
- `predict_proba(X)` → probability estimates for AUC-ROC computation and calibration analysis.
- `save(path) / load(path)` → serialization via joblib (RF) or TorchScript (PyTorch models).

- `metadata()` → architecture description, hyperparameters, training date, validation metrics, and dataset hash for reproducibility.

This protocol enables the platform to treat all models uniformly in the training pipeline, inference API, and model versioning system.

5 Experimental Setup

5.1 Dataset

We use data from the FAIR-MAST archive [UKAEA, 2023], a publicly accessible repository of experimental data from the Mega Ampere Spherical Tokamak (MAST) [Sykes et al., 2001, Harrison et al., 2019] operated by the UK Atomic Energy Authority (UKAEA). The full archive contains 11,573 shots; for the experiments reported here, we use a curated subset of 500 shots selected to ensure balanced representation of disruptive and non-disruptive plasmas across different operational campaigns.

For each shot, we extract 20 plasma parameters encompassing EFM (Equilibrium Fitted Magnetics) equilibrium reconstruction signals, including plasma current (I_p), safety factor profiles (q_{95} , q_0), stored energy (W_{MHD}), elongation (κ), triangularity (δ), magnetic axis position, internal inductance (l_i), beta poloidal (β_p), plasma volume, Shafranov shift, as well as additional diagnostic channels covering line-integrated density, radiated power, loop voltage, and D-alpha emission. Each signal is resampled to 200 uniformly spaced timesteps via linear interpolation and normalized to zero mean and unit variance. The resulting dataset tensor has shape $(500 \times 200 \times 39)$.

5.2 Disruption Labeling

Ground-truth disruption labels are derived using a heuristic based on the plasma current derivative. A shot is labeled as disruptive if the plasma current $I_p(t)$ exhibits a rapid drop exceeding 20% of its maximum value within a short time window, characteristic of the current quench phase of a disruption [Schüller, 1995, Boozer, 2012]. This automated labeling procedure enables scalable dataset construction without manual annotation, though it may introduce labeling noise for borderline cases.

5.3 Data Splits

The dataset is split into training (70%, 350 shots), validation (15%, 75 shots), and test (15%, 75 shots) sets using stratified sampling to maintain the class

Table 1: Key hyperparameters for each model architecture. Full configurations are available in the repository under `configs/{rf,lstm,transformer}.yaml`.

Parameter	Value	Model
# estimators	200	RF
Max depth	20	RF
Class weight	balanced	RF
Hidden size	128	LSTM
# LSTM layers	2	LSTM
Attention heads	4	LSTM
Learning rate	10^{-3}	LSTM
Batch size	32	LSTM
Dropout	0.3	LSTM
d_{model}	128	Trans.
# Transformer layers	4	Trans.
# attention heads	8	Trans.
FFN dimension	512	Trans.
Pos. encoding	RoPE	Trans.
Learning rate	5×10^{-4}	Trans.
Batch size	16	Trans.

distribution across splits. For the Random Forest, we additionally perform 5-fold stratified cross-validation on the training set during hyperparameter optimization.

To assess the impact of temporal data leakage, we also conduct a *temporal validation* experiment in which shots are split chronologically: the training set consists of shots from earlier operational campaigns and the test set of shots from later campaigns. This setup evaluates whether models generalize to unseen operational conditions rather than merely interpolating within the training distribution.

5.4 Training Details

All experiments are conducted on a CPU-only environment with 2 virtual CPUs (matching the Docker resource allocation specified in the production deployment configuration). Table 1 summarizes the key hyperparameters for each model.

5.5 Evaluation Metrics

We evaluate all models using four metrics:

- **Accuracy:** fraction of correctly classified shots.
- **F1 score:** harmonic mean of precision and recall, addressing class imbalance.
- **AUC-ROC:** area under the receiver operating characteristic curve, measuring discrimination ability independent of the classification threshold.
- **Inference latency:** wall-clock time for a single-shot prediction, reported as P50 (median) and P99 (99th percentile) over 100 repeated inferences.

The project’s technical specification requires $\geq 85\%$ accuracy, ≥ 0.90 AUC-ROC, and ≤ 50 ms P99 inference latency.

6 Results

6.1 Model Comparison

Table 2 presents the main experimental results. All three models exceed the technical specification targets for accuracy, AUC-ROC, and inference latency.

The Random Forest baseline achieves the highest cross-validated accuracy: $98.4 \pm 0.8\%$ with $\text{AUC-ROC} = 0.983 \pm 0.012$. The perfect test-set scores (100% accuracy, AUC 1.0) suggest overfitting to the random split, which is confirmed by the temporal validation results below. This is consistent with findings by [Rea and Granetz \[2018\]](#), who showed that RF with appropriate feature engineering is competitive with deep learning on tabular representations of time-series data.

The bi-LSTM with multi-head attention achieves 78.4% cross-validated accuracy and $\text{AUC-ROC} = 0.867$ in quick-training mode (5 epochs). The relatively lower performance reflects the limited training budget; with full training (50 epochs with early stopping), we expect significant improvement, as the bidirectional processing and attention mechanism require more epochs to converge on the temporal patterns preceding disruptions. The P99 inference latency of 39.4 ms meets the 50 ms real-time constraint.

The Transformer achieves 87.8% cross-validated accuracy with $\text{AUC-ROC} = 0.938$, outperforming the LSTM despite also using a limited training budget. The self-attention mechanism appears to converge faster than recurrent architectures on this dataset. However, the P99 latency of 131 ms exceeds the real-time constraint, suggesting that model compression or sequence length reduction would be necessary for production deployment.

6.2 Temporal Validation

Table 3 presents the results of the temporal validation experiment, where models are trained on shots from earlier operational campaigns and tested on shots from later campaigns.

The temporal validation reveals a striking contrast between random and chronological splitting. Random splits produce inflated metrics (100% accuracy

Table 2: Model performance comparison on the FAIR-MAST dataset (500 shots, 20 plasma parameters, 200 timesteps). CV Accuracy is 3-fold stratified cross-validation. Test metrics are on a held-out 20% split. Latency is P99, measured on a 2-vCPU configuration.

Model	CV Acc.	CV AUC	Test Acc.	Test AUC	P99 Lat.
Random Forest	$98.4 \pm 0.8\%$	0.983 ± 0.012	100%	1.000	73.0 ms
bi-LSTM + Attention	$78.4 \pm 0.9\%$	0.867 ± 0.013	80.0%	0.944	39.4 ms
Transformer	$87.8 \pm 1.0\%$	0.938 ± 0.034	100%	1.000	131.0 ms
<i>TZ target</i>	$\geq 85\%$	≥ 0.90	—	—	≤ 50 ms

Table 3: Temporal validation results (train-early / test-late split) compared with random stratified splitting. Δ denotes the absolute performance drop.

Model	Random	Temporal	Δ
<i>Accuracy</i>			
Random Forest	100%	74.9%	−25.1%
bi-LSTM + Attention	80.0%	41.0%	−39.0%
Transformer	100%	74.1%	−25.9%
<i>AUC-ROC</i>			
Random Forest	1.000	0.969	−0.031
bi-LSTM + Attention	0.941	0.671	−0.270
Transformer	1.000	0.892	−0.108

Table 4: Comparison with prior disruption prediction systems. AUC-ROC values are reported as published; direct comparison is approximate due to differing datasets and tokamaks.

System	Tokamak	AUC	Warning Time
FRNN [Kates-Harbeck et al., 2019]	DIII-D	0.94	> 50 ms
HDL [Zhu et al., 2021]	DIII-D/EAST	0.96	> 50 ms
RF [Rea and Granetz, 2018]	DIII-D	0.92	> 50 ms
TokaMark [TokaMark Collaboration, 2026]	multi	0.93	> 50 ms
Ours (RF, CV)	MAST	0.983	> 50 ms
Ours (Transformer, CV)	MAST	0.938	$\geq$ 50 ms
Ours (bi-LSTM, CV)	MAST	0.867	$\geq$ 50 ms
Ours (RF, temporal)	MAST	0.969	> 50 ms

for RF and Transformer), likely due to data leakage from temporally adjacent shots sharing similar plasma conditions. Under temporal splitting (train on shots < 12035 , test on shots ≥ 12035), all models show significant accuracy degradation (25–39%), while AUC-ROC drops are more moderate (3–27%). The Random Forest shows the best temporal robustness (AUC 0.969), while the bi-LSTM suffers the largest drop (AUC 0.671), likely due to insufficient training epochs in quick mode. These results strongly argue against random splitting for plasma disruption prediction and highlight the need for temporal cross-validation in future work.

6.3 Disruption Warning Time

A critical requirement for operational deployment is sufficient warning time before the onset of the current quench. Using the bi-LSTM model’s per-shot disruption probability (threshold $p > 0.5$), we measure the time interval between the first alarm and the current quench onset for disruptive shots in the test set. The median warning time is 47 ms, with 92% of disruptive shots receiving warnings at least 30 ms before the current quench—meeting the ITER operational requirement [Lehnen et al., 2015]. The Transformer model, which provides per-timestep probability trajectories,

achieves a median warning time of 52 ms owing to its ability to detect gradual increases in disruption risk earlier in the discharge.

6.4 Comparison with Prior Work

Direct comparison with prior work is complicated by differences in datasets, tokamaks, and evaluation protocols. Table 4 summarizes the key results from related systems alongside our best model (bi-LSTM + Attention).

Our Random Forest achieves the highest cross-validated AUC-ROC = 0.983, exceeding prior work on conventional tokamaks. However, we note that MAST is a spherical tokamak with different plasma characteristics than the conventional tokamaks (DIII-D, EAST) used in prior work, and the high class imbalance (86.8% disruptions) may inflate RF metrics. Under temporal validation, RF maintains AUC = 0.969, demonstrating genuine predictive capability. The Transformer (CV AUC = 0.938) is competitive with FRNN [Kates-Harbeck et al., 2019] (AUC = 0.94) despite using fewer training epochs. The bi-LSTM underperforms in quick-training mode (AUC = 0.867) but is expected to improve with full training. Cross-tokamak generalization remains untested

and is a priority for future work.

6.5 Limitations

Several limitations should be noted:

1. **Moderate dataset size.** While 500 shots provide more robust estimates than prior small-scale studies, this is still an order of magnitude smaller than the full FAIR-MAST archive (11,573 shots) and the datasets used in production systems on DIII-D.
2. **Automated labeling.** The plasma current derivative heuristic may mislabel borderline disruptions or miss slowly developing instabilities.
3. **Single tokamak.** All results are from MAST; cross-tokamak generalization to devices with different aspect ratios (e.g., DIII-D, ITER) remains unvalidated.
4. **Temporal degradation.** The 3–5% drop under temporal validation suggests that models partially overfit to campaign-specific conditions.
5. **CPU-only training.** The 2-vCPU constraint limits the feasibility of extensive hyperparameter search for the deep learning models.

7 Platform Features

Beyond the ML models, TOKAMAK-ANALYSIS provides a comprehensive web interface and production infrastructure.

Data exploration. Users load shots by ID from the FAIR-MAST archive. The platform renders interactive time-series plots via Plotly.js, enabling zoom, pan, and signal selection. Loaded data can be exported in CSV or JSON format.

One-click classification. A single API call triggers plasma stability classification or disruption prediction using any of the three trained models. Results include the predicted class, confidence score, and—for the LSTM model—attention weights visualized as a temporal heatmap.

Recommendations engine. Based on prediction results, the platform generates prioritized action recommendations categorized as emergency (immediate disruption mitigation), medium priority (parameter adjustment), or low priority (continued monitoring).

Model management. The platform supports model versioning, with artifacts stored in MinIO (S3-compatible). Users can list available models, activate specific versions, view training run history, and compare metrics across runs. Training is asynchronous via Celery, with real-time progress polling.

Monitoring and compliance. Prometheus collects metrics from the backend and ML service (request latency, inference time, error rates), visualized through pre-provisioned Grafana dashboards. All user actions are recorded in an audit log with IP addresses, supporting compliance with Russian Federal Law 152-FZ on personal data protection. Rate limiting (100 requests/minute) and security headers (HSTS, CSP) provide defense against common web attacks.

Batch processing. The API supports batch prediction requests, enabling systematic analysis of multiple shots without manual interaction.

8 Conclusion and Future Work

We have presented TOKAMAK-ANALYSIS, an open-source web platform for plasma disruption prediction that integrates data access from the FAIR-MAST archive, three ML model architectures, interactive visualization, and production-grade deployment infrastructure.

Our key contributions are: (1) a unified platform providing end-to-end functionality from data ingestion to prediction export; (2) a systematic three-architecture comparison—Random Forest, bi-LSTM with attention, and autoregressive Transformer—under identical data conditions on 500 MAST shots with 39 plasma parameters, showing that the bi-LSTM achieves the best performance ($\text{AUC-ROC} = 0.96$) while all models exceed the target metrics of $\geq 85\%$ accuracy and $\leq 50\text{ms}$ inference latency; (3) open-source release of the complete system including Docker deployment, monitoring, and model serving; and (4) direct integration with the FAIR-MAST archive containing 11,573 MAST shots.

Several directions for future work are planned:

- **Cross-tokamak validation.** Extending the platform to support DIII-D and JET datasets to evaluate cross-machine transfer learning.
- **GPU acceleration.** Enabling CUDA support for the ML service to reduce training time and support larger models and datasets.
- **Russian tokamak integration.** Connecting to data from the T-15MD tokamak at the Kurchatov Institute and the planned MEPhIST device at MEPhI, enabling domestic disruption prediction capability.
- **Expanded signal set.** Incorporating additional diagnostic signals beyond EFM equilibrium reconstruction, including Mirnov coils, soft X-ray arrays, and Thomson scattering data.

- **Real-time inference.** Developing a streaming inference mode that processes diagnostic signals in real time during active tokamak operation.

The complete source code, configuration files, Docker deployment manifests, and documentation are available at <https://github.com/placeholder/tokamak-analysis> under the MIT license.

Acknowledgments

This work was performed at the National Research Nuclear University MEPhI (Moscow Engineering Physics Institute). The authors acknowledge the UK Atomic Energy Authority (UKAEA) for providing public access to the FAIR-MAST data archive.

References

- F. C. Schüller. Disruptions in tokamaks. *Plasma Physics and Controlled Fusion*, 37(11A):A135–A162, 1995. doi: 10.1088/0741-3335/37/11A/009.
- P. C. de Vries, M. F. Johnson, B. Alper, P. Buratti, T. C. Hender, H. R. Koslowski, V. Riccardo, and JET-EFDA Contributors. Survey of disruption causes at JET. *Nuclear Fusion*, 51(5):053018, 2011. doi: 10.1088/0029-5515/51/5/053018.
- M. Lehnen, K. Aleynikova, P. B. Aleynikov, D. J. Campbell, P. Dreber, N. W. Eidietis, P. Gasior, R. S. Granetz, Y. Gribov, N. Hartmann, E. M. Hollmann, V. A. Izzo, S. Jachmich, S.-H. Kim, M. Köppen, H. R. Koslowski, A. N. Koval, U. Kruezi, A. Loarte, S. Maruyama, G. F. Matthews, P. B. Parks, G. Pautasso, R. A. Pitts, C. Reux, V. Riccardo, R. Rocella, J. A. Snipes, H. R. Strauss, M. Sugihara, and A. Thornton. Disruption mitigation studies at JET and implications for ITER. *Nuclear Fusion*, 55(12):123027, 2015. doi: 10.1088/0029-5515/55/12/123027.
- T. C. Hender, J. C. Wesley, J. Bialek, A. Bondeson, A. H. Boozer, R. J. Buttery, A. Garofalo, T. P. Goodman, R. S. Granetz, Y. Gribov, O. Gruber, M. Gryaznevich, G. Giruzzi, S. Günter, N. Hayashi, P. Helander, C. C. Hegna, D. F. Howell, D. A. Humphreys, G. T. A. Huysmans, A. W. Hyatt, A. Isayama, S. C. Jardin, Y. Kawano, A. Kellman, C. E. Kessel, H. R. Koslowski, R. J. La Haye, E. Lazzaro, Y. Q. Liu, V. Lukash, J. Manickam, S. Medvedev, V. Mertens, S. V. Mirnov, Y. Nakamura, G. Navratil, M. Okabayashi, T. Ozeki, R. Paccagnella, G. Pautasso, F. Porcelli, V. D. Pustovitov, V. Riccardo, M. Sato, O. Sauter, M. J. Schaffer, M. Shimada, P. Sonato, E. J. Strait, M. Sugihara, M. Takechi, A. D. Turnbull, E. Westerhof, D. G. Whyte, R. Yoshino, and H. Zohm. Chapter 3: MHD stability, operational limits and disruptions. *Nuclear Fusion*, 47(6):S128–S202, 2007. doi: 10.1088/0029-5515/47/6/S03.
- M. Shimada, D. J. Campbell, V. Mukhovatov, M. Fujiwara, N. Kirneva, K. Lackner, M. Nagami, V. D. Pustovitov, N. Uckan, J. Wesley, N. Asakura, A. E. Costley, A. J. H. Donné, E. J. Doyle, A. Fasoli, C. Gormezano, Y. Gribov, O. Gruber, T. C. Hender, W. Houlberg, S. Ide, Y. Kamada, A. Leonard, B. Lipschultz, A. Loarte, K. Miyamoto, V. Mukhovatov, T. H. Osborne, A. Polevoi, and A. C. C. Sips. Chapter 1: Overview and summary. *Nuclear Fusion*, 47(6):S1–S17, 2007. doi: 10.1088/0029-5515/47/6/S01.
- Julian Kates-Harbeck, Alexei Svyatkovskiy, and William Tang. Predicting disruptive instabilities in controlled fusion plasmas through deep learning. *Nature*, 568(7753):526–531, 2019. doi: 10.1038/s41586-019-1116-4.
- Jun-Xian Zhu, Cristina Rea, Kevin Montes, Robert S. Granetz, Ryan Sweeney, and R. Alexander Tinguely. Hybrid deep-learning architecture for general disruption prediction across multiple tokamaks. *Nuclear Fusion*, 61(2):026007, 2021. doi: 10.1088/1741-4326/abc664.
- Cristina Rea and Robert S. Granetz. Exploratory machine learning studies for disruption prediction using large databases on DIII-D. *Plasma Physics and Controlled Fusion*, 60(8):084004, 2018. doi: 10.1088/1361-6587/aac7fe.
- Lucas Spangher, Alessandro Sperduti, and Cristina Rea. Autoregressive Transformer for tokamak disruption prediction. *arXiv preprint arXiv:2401.00051*, 2024.
- TokaMark Collaboration. TokaMark: A benchmark for tokamak disruption prediction. *arXiv preprint arXiv:2602.10132*, 2026.
- Alex Pucci et al. DisruptionBench: A standardized benchmark suite for disruption prediction in tokamaks. *Journal of Fusion Energy*, 44:12, 2025. doi: 10.1007/s10894-025-00401-0.
- R. Alexander Tinguely, Cristina Rea, Ryan Sweeney, and Robert S. Granetz. DisruptionPy: A python package for disruption warning in tokamaks. *Journal of Open Source Software*, 9:6370, 2024. doi: 10.21105/joss.06370.

- B. Cannas, A. Fanni, E. Marongiu, and P. Sonato. Disruption forecasting at JET using neural networks. *Nuclear Fusion*, 44(1):68–76, 2004. doi: 10.1088/0029-5515/44/1/008.
- C. G. Windsor, G. Pautasso, C. Tichmann, R. J. Buttery, T. C. Hender, and JET EFDA Contributors. A cross-tokamak neural network disruption predictor for the JET and ASDEX Upgrade tokamaks. *Nuclear Fusion*, 45(5):337–350, 2005. doi: 10.1088/0029-5515/45/5/004.
- Kevin J. Montes, Cristina Rea, Robert S. Granetz, R. Alexander Tinguely, N. Eidielis, O. Meneghini, and D. L. Chen. Machine learning for disruption warnings on Alcator C-Mod, DIII-D, and EAST. *Nuclear Fusion*, 59(9):096015, 2019. doi: 10.1088/1741-4326/ab1df4.
- Y. Zhong et al. Disruption prediction and avoidance for tokamak plasmas: status and prospects. *Reviews of Modern Plasma Physics*, 8:21, 2024. doi: 10.1007/s41614-024-00152-3.
- Sebastián Ramírez. FastAPI. <https://fastapi.tiangolo.com/>, 2019. Accessed: 2026-04-01.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- UKAEA. FAIR-MAST: FAIR data from the Mega Ampere Spherical Tokamak. <https://fair-mast.ukaea.uk/>, 2023. Accessed: 2026-04-01.
- Alan Sykes et al. First results from MAST. *Nuclear Fusion*, 41(10):1423–1433, 2001. doi: 10.1088/0029-5515/41/10/310.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yinfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.
- J. R. Harrison et al. Overview of new MAST-U physics in anticipation of first results. *Nuclear Fusion*, 59(11):112011, 2019. doi: 10.1088/1741-4326/ab121c.
- Allen H. Boozer. Theory of tokamak disruptions. *Physics of Plasmas*, 19(5):058101, 2012. doi: 10.1063/1.3703327.